

TP2 : Strategie de Deploiement Blue/Green

Introduction : Qu'est-ce que le Blue/Green ?

L'analogie du restaurant

Imaginez un restaurant avec DEUX cuisines identiques : la cuisine **Bleue** et la cuisine **Verte**.

- **Aujourd'hui** : la cuisine Bleue prepare les plats pour les clients.

La cuisine Verte est vide, elle attend.

- **Vous voulez changer le menu** : au lieu de fermer le restaurant pour modifier la cuisine Bleue (ce qui priverait les clients), vous preparez le nouveau menu dans la cuisine Verte pendant que la Bleue continue de servir les clients.

- **Le nouveau menu est pret** : vous basculez ! Les serveurs vont maintenant chercher les plats dans la cuisine Verte. Les clients ne se sont rendu compte de rien.

- **Un probleme avec le nouveau menu ?** Pas de panique ! Vous rebasculez les serveurs vers la cuisine Bleue en 2 secondes. L'ancien menu fonctionne toujours parfaitement.

C'est exactement le principe du **deploiement Blue/Green** !

Le schema

```

ETAT INITIAL :
[Blue: v1.0] <==== Traefik =====> Utilisateurs
[Green: ---]      (routeur)

DEPLOIEMENT DE LA NOUVELLE VERSION :
[Blue: v1.0] <==== Traefik =====> Utilisateurs
[Green: v2.0]      (routeur)          (toujours sur Blue)

BASCULE :
[Blue: v1.0]      Traefik =====> Utilisateurs
[Green: v2.0] <===  (routeur)          (maintenant sur Green !)

EN CAS DE PROBLEME (ROLLBACK) :
[Blue: v1.0] <==== Traefik =====> Utilisateurs (retour sur Blue)
[Green: v2.0]      (routeur)

```

Avantages du Blue/Green

Avantage	Explication
Zero downtime	Les utilisateurs n'ont aucune coupure de service
Rollback instantane	On rebascule vers l'ancienne version en 1 seconde
Test avant bascule	On peut tester Green avant de basculer le trafic
Simplicité	Concept facile a comprendre et a implementer

Inconvenients du Blue/Green

Inconvenient	Explication
Double de ressources	Il faut deux environnements complets (Blue + Green)
Base de donnees	Les migrations de BD doivent etre compatibles avec les deux versions
Cout	Plus de serveurs = plus de cout en infrastructure

Objectifs du TP

A la fin de ce TP, vous serez capable de :

1. Comprendre le principe du deploiement Blue/Green
2. Configurer Traefik comme reverse proxy / routeur
3. Creer deux environnements Docker (Blue et Green)
4. Basculer le trafic entre Blue et Green
5. Ecrire des smoke tests pour verifier le deploiement
6. Implementer un rollback automatique

Prerequis

Avant de commencer, vérifiez que vous avez :

- [] **Docker** installe : tapez `docker --version` dans le terminal. Si vous voyez un numéro de version (ex: `Docker version 24.0.6`), c'est bon. Sinon, installez Docker Desktop depuis <https://www.docker.com/products/docker-desktop/>
- [] **Docker Compose** installe : tapez `docker compose version` dans le terminal. Si vous voyez un numéro de version, c'est bon. Docker Compose est inclus dans Docker Desktop.
- [] Avoir complété le **TP1**
- [] Un compte GitLab avec un projet

****Comment ouvrir un terminal ?****

- ****Windows**** : clic droit sur le Bureau > "Ouvrir dans le terminal", ou cherchez "Git Bash" dans le menu Démarrer
- ****Mac**** : ouvrez l'application "Terminal"
- ****Linux**** : Ctrl+Alt+T

Architecture du TP

Voici comment les éléments vont communiquer entre eux :

```
Navigateur --> Traefik (port 80) --> Blue OU Green
```

```
Traefik = Le "routeur" qui decide vers quel environnement envoyer le trafic
Blue    = Conteneur Docker avec l'ancienne version de l'application
Green   = Conteneur Docker avec la nouvelle version de l'application
```

****En résumé**** : Traefik est comme un aiguillage de train. Il reçoit les requêtes des utilisateurs et les envoie vers Blue ou Green selon la configuration.

Structure des fichiers

Tous les fichiers nécessaires sont **déjà fournis** dans le dossier de ce TP.

Vous n'avez PAS besoin de les créer vous-même.

```
tp2-blue-green/
.gitlab-ci.yml          <-- Pipeline Blue/Green (automatisation)
docker-compose.blue.yml <-- Configuration de l'environnement Blue
docker-compose.green.yml <-- Configuration de l'environnement Green
traefik.yml             <-- Configuration du routeur Traefik
smoke-tests.sh          <-- Script de vérification automatique
GUIDE-TP2.md            <-- Ce document que vous lisez
```

Etape 1 : Comprendre Traefik

Qu'est-ce que Traefik ?

Traefik est un **reverse proxy** (proxy inverse). Son rôle est de recevoir les requêtes des utilisateurs et de les **rediriger** vers le bon serveur.

```
Utilisateur --> Traefik --> Serveur Blue (ou Green)
```

****Analogie**** : Traefik est comme un standardiste téléphonique. Il reçoit les appels (requêtes) et les redirige vers le bon bureau (serveur). Quand on veut changer de bureau (passer de Blue à Green), on change simplement les instructions du standardiste, sans couper les appels.

Pourquoi Traefik pour le Blue/Green ?

Traefik peut changer la redirection **EN DIRECT**, sans redémarrer.

C'est ce qui permet la bascule instantanée entre Blue et Green.

Les utilisateurs ne voient aucune coupure.

Ce que vous devez faire

1. Ouvrez le fichier **traefik.yml** dans votre éditeur de texte
2. Lisez **chaque commentaire** pour comprendre ce que fait chaque ligne
3. Pas besoin de modifier ce fichier, il est prêt à l'emploi

Le fichier **traefik.yml** configure :

- Un **point d'entrée sur le port 80** : c'est là que les requêtes HTTP arrivent
- Un **point d'entrée sur le port 8080** : c'est le tableau de bord (dashboard) de Traefik, pour voir ce qui se passe
- La **découverte automatique des services Docker** : Traefik détecte automatiquement les conteneurs Docker qui démarrent ou s'arrêtent

Etape 2 : Comprendre les fichiers Docker Compose

Qu'est-ce que Docker Compose ?

Docker Compose permet de définir et lancer plusieurs conteneurs Docker à partir d'un fichier YAML. Chaque conteneur est un "service".

****Analogie**** : Docker Compose est comme une recette de cuisine qui liste tous les ingrédients (conteneurs) nécessaires et comment les préparer.

Ce que vous devez faire

1. Ouvrez le fichier `docker-compose.blue.yml` dans votre éditeur de texte et lisez-le
2. Ouvrez le fichier `docker-compose.green.yml` dans votre éditeur de texte et lisez-le
3. **Comparez les deux fichiers** : vous verrez qu'ils sont presque identiques

Les différences entre Blue et Green

Element	docker-compose.blue.yml	docker-compose.green.yml
Nom du conteneur	<code>app-blue</code>	<code>app-green</code>
Labels Traefik	Identifie comme "Blue"	Identifie comme "Green"

Tout le reste est identique. C'est le principe du Blue/Green : deux environnements **identiques** sauf le nom.

Etape 3 : Lancer l'infrastructure

Ce que vous allez faire

Démarrer Traefik (le routeur) et l'environnement Blue (la version actuelle de l'application).

Instructions pas à pas

Ouvrez un terminal et **placez-vous dans le dossier du TP2** :

```
cd chemin/vers/tp2-blue-green
```

****Remplacez**** ``chemin/vers/`` par le vrai chemin sur votre machine.
Par exemple : ``cd ~/Desktop/CI\ CD/jour1/tp2-blue-green``

****IMPORTANT**** : Assurez-vous que Docker Desktop est lancé avant de continuer.
Sur Windows/Mac, ouvrez l'application Docker Desktop et attendez qu'elle soit prête (l'icône dans la barre des tâches doit être stable, sans animation).

Exécutez ces commandes **une par une** :

```
# COMMANDE 1 : Creer un reseau Docker
# Un reseau Docker permet aux conteneurs de communiquer entre eux.
# Le reseau s'appelle "web" et sera partage entre Traefik, Blue et Green.
docker network create web
```

Si vous voyez l'erreur "network with name web already exists", c'est que le reseau existe deja. Ce n'est pas un probleme, passez a la commande suivante.

```
# COMMANDE 2 : Lancer Traefik (le routeur)
# -f = specifier quel fichier Docker Compose utiliser
# up = demarrer les conteneurs
# -d = en arriere-plan (detached) : le terminal reste libre
docker compose -f traefik.yml up -d
```

```
# COMMANDE 3 : Lancer l'environnement Blue (version actuelle de l'application)
docker compose -f docker-compose.blue.yml up -d
```

```
# COMMANDE 4 : Verifier que tout fonctionne
# Cette commande liste tous les conteneurs Docker en cours d'execution
docker ps
```

Resultat attendu

La commande **docker ps** doit afficher **2 conteneurs** en cours d'execution :

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
abc123	traefik:v2.10	Up	0.0.0.0:80->80/tcp	traefik
def456	node:18-alpine	Up	3000/tcp	app-blue

****Si vous ne voyez pas les 2 conteneurs**** : attendez quelques secondes et relancez ``docker ps``.
Les conteneurs peuvent mettre un moment a demarrer.

Tester que l'application repond

```
# Test 1 : Verifier que l'application est en vie (health check)
curl http://localhost/health
```

Reponse attendue (un objet JSON) :

```
{"status":"ok","timestamp":"2024-...", "uptime":...}
```

```
# Test 2 : Verifier la liste des utilisateurs
curl http://localhost/api/users
```

Reponse attendue (un tableau JSON) :

```
[{"id":1,"fullName":"Jean DUPONT","age":30}, {"id":2,"fullName":"Marie MARTIN","age":25}, {"id":3,"fullName":"P
```

****Si `curl` n'est pas reconnu**** : sur Windows, utilisez PowerShell avec la commande
`Invoke-WebRequest http://localhost/health` ou ouvrez simplement votre navigateur
et tapez `http://localhost/health` dans la barre d'adresse.

Etape 4 : Deployer la nouvelle version (Green)

Ce que vous allez faire

Lancer l'environnement Green (la nouvelle version) **a cote** de Blue.

A ce stade, les deux versions tournent en meme temps, mais les utilisateurs continuent d'utiliser Blue.

Instructions pas a pas

```
# COMMANDE 1 : Lancer l'environnement Green
# A ce stade, Blue ET Green tournent en meme temps
# Mais Traefik envoie encore tout le trafic vers Blue
docker compose -f docker-compose.green.yml up -d
```

```
# COMMANDE 2 : Verifier que les DEUX environnements tournent
docker ps
```

Resultat attendu

La commande **docker ps** doit maintenant afficher **3 conteneurs** :

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
abc123	traefik:v2.10	Up	80, 8080	traefik
def456	node:18-alpine	Up	3000/tcp	app-blue
ghi789	node:18-alpine	Up	3000/tcp	app-green

****Vous devez voir 3 lignes**** : traefik, app-blue ET app-green.
Si app-green n'apparaît pas, relancez la commande 1.

Etape 5 : Tester Green avant la bascule

Pourquoi tester avant ?

C'est l'un des **GRANDS avantages** du Blue/Green : on peut tester la nouvelle version **AVANT** de basculer le trafic.

Si Green ne fonctionne pas, on ne bascule pas et les utilisateurs ne sont jamais impactés.

Instructions pas a pas

```
# Lancer les smoke tests sur l'environnement Green
# Le script smoke-tests.sh verifie automatiquement que les endpoints repondent correctement
bash smoke-tests.sh green
```

****Qu'est-ce qu'un "smoke test" ?**** C'est un test rapide et basique qui vérifie que l'application démarre correctement et répond aux requêtes. C'est comme allumer une machine pour vérifier qu'elle ne "fume" pas (d'où le nom "smoke test").

Resultat attendu

```
=== SMOKE TESTS ===
[OK] Health check : status ok
[OK] API Users : 3 utilisateurs retournes
[OK] API User/1 : utilisateur trouve
=== TOUS LES TESTS PASSES ===
```

****Si les tests échouent**** :

- Vérifiez que le conteneur `app-green` est bien en cours d'exécution avec `docker ps`
- Attendez quelques secondes que le conteneur finisse de démarrer
- Relancez les smoke tests

Etape 6 : Basculer le trafic vers Green

Ce que vous allez faire

Arrêter l'environnement Blue pour que Traefik redirige automatiquement tout le trafic vers Green. C'est LA bascule !

Instructions pas a pas

```
# COMMANDE 1 : Arrêter l'environnement Blue
# Traefik detecte automatiquement que Blue n'est plus disponible
# et redirige TOUT le trafic vers Green
docker compose -f docker-compose.blue.yml stop
```

```
# COMMANDE 2 : Verifier que l'application repond TOUJOURS
# Meme si Blue est arrete, Green prend le relais
curl http://localhost/health
```

Reponse attendue : `{"status":"ok","timestamp":"...","uptime":...}`

```
# COMMANDE 3 : Verifier que les utilisateurs sont toujours accessibles
curl http://localhost/api/users
```

Reponse attendue : `[{"id":1,"fullName":"Jean DUPONT",...},...]`

Resultat attendu

L'application repond **toujours** ! Les utilisateurs n'ont eu **aucune** interruption de service. Le trafic passe maintenant par Green au lieu de Blue.

****C'est la magie du Blue/Green**** : la bascule est quasi-instantanee et transparente pour les utilisateurs.

Verifier avec docker ps

```
docker ps
```

Vous devriez voir **2 conteneurs** (Blue n'est plus la) :

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
abc123	traefik:v2.10	Up	80, 8080	traefik
ghi789	node:18-alpine	Up	3000/tcp	app-green

Etape 7 : Simuler un rollback

Scenario

Imaginons que Green a un probleme (un bug, un crash, des performances degradees).

On veut revenir a Blue **immédiatement**, sans perdre de temps.

Instructions pas a pas

```
# COMMANDE 1 : Relancer l'environnement Blue (l'ancienne version, qui fonctionnait bien)
docker compose -f docker-compose.blue.yml up -d
```

```
# COMMANDE 2 : Arrêter l'environnement Green (celui qui a le probleme)
docker compose -f docker-compose.green.yml stop
```

```
# COMMANDE 3 : Verifier que l'application fonctionne avec Blue
curl http://localhost/health
```

Reponse attendue : `{"status":"ok","timestamp":"...","uptime":...}`

Resultat attendu

L'application fonctionne a nouveau avec la version Blue.

Le rollback a pris **quelques secondes seulement** !

****Comparez avec un deploiement classique**** : si on avait remplace directement l'ancienne version par la nouvelle, et que la nouvelle avait un bug, il faudrait reconstruire et redeployer l'ancienne version. Cela prendrait des minutes, voire des heures. Avec Blue/Green, c'est quelques secondes.

Etape 8 : Comprendre le pipeline GitLab CI

Ce que vous allez faire

Lire et comprendre le fichier `.gitlab-ci.yml` de ce TP, qui automatise toutes les etapes que vous venez de faire manuellement.

Instructions pas a pas

1. Ouvrez le fichier `.gitlab-ci.yml` du dossier `tp2-blue-green` dans votre editeur
2. Lisez-le en entier, avec tous les commentaires

3. Identifiez les 5 stages du pipeline :

Stage	Ce qu'il fait	Automatique ou Manuel ?
build	Construit l'image Docker de la nouvelle version	Automatique
deploy-green	Deploie la nouvelle version sur l'environnement Green	Automatique
smoke-test	Teste Green automatiquement (les memes smoke tests que vous avez lances)	Automatique
switch	Bascule le trafic de Blue vers Green	MANUEL (clic requis)
rollback	Revient a Blue en cas de probleme	MANUEL (clic requis)

Points clés à retenir

- La bascule (switch) est **manuelle** (**when: manual**) : un humain doit valider avant de basculer le trafic. C'est une securite.
- Le rollback est **toujours disponible** : meme apres la bascule, on peut revenir en arriere a tout moment.
- Les smoke tests s'executent **AVANT** la bascule : on ne bascule que si tout est vert. Si les smoke tests echouent, la bascule n'est meme pas proposee.

Etape 9 : Nettoyage

Ce que vous allez faire

Arreter tous les conteneurs Docker et supprimer le reseau. C'est important pour liberer les ressources de votre machine.

Instructions pas a pas

Executez ces commandes **une par une** :

```
# COMMANDE 1 : Arreter et supprimer les conteneurs Blue
# "down" arrete ET supprime les conteneurs (contrairement a "stop" qui les arrete seulement)
docker compose -f docker-compose.blue.yml down
```

```
# COMMANDE 2 : Arreter et supprimer les conteneurs Green
docker compose -f docker-compose.green.yml down
```

```
# COMMANDE 3 : Arreter et supprimer Traefik
docker compose -f traefik.yml down
```

```
# COMMANDE 4 : Supprimer le reseau Docker "web" cree a l'etape 3
docker network rm web
```

Verification

```
# Verifier qu'il ne reste aucun conteneur du TP
docker ps
```

Vous ne devriez plus voir les conteneurs **traefik**, **app-blue** ou **app-green**.

Si des conteneurs persistent, forcez leur arret avec : ``docker stop traefik app-blue app-green``

Troubleshooting (Resolution de problemes)

Probleme 1 : "network web not found"

Quand ca arrive : quand vous lancez un conteneur mais le reseau Docker "web" n'existe pas encore

Solution : creez le reseau avec la commande `docker network create web`

Probleme 2 : "port 80 already in use" (port 80 deja utilise)

Quand ca arrive : quand vous lancez Traefik mais un autre programme utilise deja le port 80

Cause : un autre serveur web (Apache, Nginx, IIS, Skype, etc.) utilise deja le port 80

Solution :

- Arretez l'autre programme qui utilise le port 80
- OU changez le port dans `traefik.yml` : remplacez `"80:80"` par `"8081:80"`, puis utilisez `http://localhost:8081` au lieu de `http://localhost`

Probleme 3 : Les conteneurs ne communiquent pas entre eux

Quand ca arrive : quand Traefik ne redirige pas vers Blue ou Green

Solution : verifiez que tous les conteneurs sont sur le meme reseau Docker :

```
docker network inspect web
```

Vous devriez voir les 3 conteneurs (traefik, app-blue, app-green) dans la section "Containers".

Probleme 4 : "curl: connection refused"

Quand ca arrive : quand vous testez avec curl mais le conteneur n'est pas encore pret

Solution :

- Attendez 5-10 secondes que les conteneurs finissent de demarrer
- Verifiez que les conteneurs sont bien en cours d'execution avec `docker ps`
- Si un conteneur a le statut "Exited", c'est qu'il a plante. Regardez ses logs avec :

```
docker logs app-blue
```

Probleme 5 : Traefik ne route pas le trafic

Quand ca arrive : quand l'application ne repond pas malgre les conteneurs en cours d'execution

Solution :

- Ouvrez le dashboard Traefik dans votre navigateur : `http://localhost:8080`
- Vous devriez voir les services Blue et/ou Green enregistres dans l'onglet "HTTP Services"
- Si aucun service n'apparait, verifiez les labels Docker dans les fichiers `docker-compose.*.yaml`

Checklist de validation

Cochez chaque element quand vous l'avez complete :

- ☐ Je comprends le principe du deploiement Blue/Green (deux environnements identiques, bascule instantanee)
- ☐ Je comprends le role de Traefik (routeur / reverse proxy qui redirige le trafic)
- ☐ J'ai lance Traefik et l'environnement Blue (etape 3)
- ☐ J'ai verifie que l'application repond sur `http://localhost` (avec curl ou le navigateur)
- ☐ J'ai lance l'environnement Green a cote de Blue (etape 4)
- ☐ J'ai execute les smoke tests sur Green et ils sont passes (etape 5)
- ☐ J'ai bascule le trafic vers Green en arretant Blue (etape 6)
- ☐ J'ai verifie que l'application repond toujours apres la bascule
- ☐ J'ai simule un rollback vers Blue (etape 7)
- ☐ J'ai lu et compris le pipeline GitLab CI du Blue/Green (etape 8)
- ☐ J'ai nettoye l'environnement avec `docker down` (etape 9)

Pour aller plus loin

Si vous avez termine avant les autres, essayez :

1. **Ajoutez une base de donnees** : Comment gerer les migrations de BD avec Blue/Green ? (Indice : les migrations doivent etre retrocompatibles)
2. **Implementez un health check Docker** : Ajoutez un `HEALTHCHECK` dans le Dockerfile pour que Docker verifie automatiquement la sante du conteneur

3. Ajoutez des metriques : Utilisez Prometheus et Grafana pour surveiller les performances avant et apres la bascule

4. Automatisez completement : Modifiez le pipeline pour que la bascule soit automatique si les smoke tests passent

****Felicitations !**** Vous avez complete le TP2. Vous savez maintenant deployer une application avec la strategie Blue/Green, la methode la plus sure pour des deploiements sans interruption de service.